   a. *List[i]*, which returns the element present at the i th node of the list.
   b. *InsertList(value, i)*, which inserts the new element as the *i* th node in the list.
   c. *DeleteList(i)*, which deletes the i th node of the list.
6. Given an array of integers *S*, and a number *N*, can you print the combination of numbers from *S* which sum up to *N*? Note that we want the combination, and not the permutation. For example, given *S* as 1,2,3,6,2,4,1 and *N* as 8, one combination we can print out is (1,2,3,2), and if we print out this combination, we should not print out (2,3,2,1), etc.
7. Given a string *S*, a three-letter pattern *xyz*, and a character *D*, write an algorithm to replace all occurrences of the pattern *xyz* in *S* by *D*, in-place. Note that if multiple copies of the pattern *xyz* occur together, they should be replaced by a single *D*. For instance: if you have *axyzxyzb*, then after replacement, the string should be *aDb*.
8. Given a linked list *L*, which consists of *N* nodes, split this into two lists: *L1*, which contains the front part of *L*, and *L2*, which contains the back part of *L*. *L1* and *L2* each contain half the elements of *L*. If *N* is odd, add the extra element to *L1*.
   a. What is the complexity of your algorithm?
   b. Can you do this in a single list traversal?
9. What is the difference between the keyword *new* and the operator *new* in C++?
10. In concurrent programming, what is the difference between wait freedom and lock freedom? Which one provides a stronger forward-progress guarantee?
11. You are given an array of integers *N*, such that each integer in it appears thrice (i.e., there are triplicates for every element) except one integer. Can you find the single integer that does not have copies in the array?
12. You are given a set of unsorted integers, which is represented by a linked list. Each node of the list contains an integer. If you are asked to sort the set of integers, what approach would you use, and why?
13. Can you explain what a NoSQL database is? Do they provide ACID guarantees?
14. What is the difference between the super-scalar and VLIW architectures? And what is the difference between in-order and out-of-order processors? Does super-scalar imply out-of-order execution?
15. Can you differentiate between an archive library and a shared library? When would you prefer to link an archive library with your application, and when would you prefer to use a shared library?
16. Can you write a handler for a *SIGILL* signal? When is *SIGILL* typically encountered?
17. Pages are typically of size 4K and above. Why is the page size as high as 4K? Why can't we have a page size of 128 bytes? What are the pros and cons of having a small page size?
18. Do you know the difference between simultaneous multi-threading and hyper-threading?
19. What is the significance of the memristor, phase-change memory and spin-torque-transfer RAM technologies to the computing paradigm?
20. What is the difference between actor-based programming and thread-based programming?
21. What are the various code-coverage measurements you would do to make sure that your code is tested adequately?
22. Do you know what is meant by cyclomatic complexity?
23. Can you explain how a breakpoint is implemented in a debugger?
24. What is ROP or return-oriented programming?
25. What are lambda expressions in C++?

This month's takeaway question

Since there were no correct solutions from our readers for last month's takeaway question on calling *delete* on a *void** pointer, I would like to keep the question open for this month as well. Please do send me your answers. Remember that we are invoking the C++ keyword *delete* with a *void** pointer, in the problem given in last month's column. What would happen in that case? Think about what the difference is between *delete* and the operator *delete*.

My 'must-read book' for this month

This month's 'must-read book' is actually one that I came across as an online version. It is called '97 Things A Programmer Should Know', from O'Reilly. The contributions published in the book are available at the wiki, *http://programmer.97things.oreilly.com/wiki/index.php/Contributions_Appearing_in_the_Book*.

Though some of the contributions are simple, common-sense approaches to programming, it makes an interesting read, since a wide spectrum of topics are covered, right from coding errors to re-factoring. If you have a favourite programming book that you think is a must-read for every programmer, please do send me a note with the book's name, and a short write-up on why you think it is useful, so I can mention it in this column. It would help many readers who want to improve their coding skills.

If you have any favourite programming puzzles that you would like to discuss on this forum, please send them to me, along with your solutions and feedback, at *sandyasm_AT_yahoo_DOT_com*. Till we meet again next month, happy programming, and here's wishing you the very best! END

By: Sandya Mannarswamy

The author is an expert in systems software, and works at Hewlett-Packard India. Her interests include compilers, virtualisation technologies and software development tools. If you are preparing for systems software interviews, you may find it useful to visit Sandya's website *http://www.tech-pathshala.com*, which offers practice interviews and training for systems software jobs. You can visit her LinkedIn group tech-pathshala's system software training group at *http://www.linkedin.com/groups?home=&gid=3813966&trk=anet_ug_hm*.